

Real Estate & Credit Risk Engine

A Complete Theoretical Walkthrough

Combining Unsupervised Clustering · Regression · Classification
in an End-to-End Scikit-Learn Pipeline

P R E S E N T E R S

Problem Statement & Overview

M
2

Data Engineering & Preprocessing

M
3

Unsupervised Learning – KMeans

M
4

Supervised Learning – Linear Regression

M
5

Classification – Random Forest

M
6

System Integration & Pipeline

M
7

Results, Evaluation & Conclusions

Presentation Agenda

01

Problem Statement & Overview

Member 1 · ~5 min

02

Data Engineering & Preprocessing

Member 2 · ~5 min

03

Unsupervised Learning – KMeans Clustering

Member 3 · ~5 min

04

Supervised Learning – Linear Regression

Member 4 · ~5 min

05

Classification – Random Forest

Member 5 · ~5 min

06

System Integration & Pipeline

Member 6 · ~5 min

07

Results, Evaluation & Conclusions

Member 7 · ~5 min

M1

Member 1

Problem Statement & Overview

Why this system? What problem does it solve?

M1 • The Problem

Problem 1

Property Valuation Gap

Real estate prices depend on dozens of interacting factors — structural features AND neighborhood socioeconomics. Traditional manual appraisals miss complex non-linear patterns.

Problem 2

Credit Default Risk

Lenders must assess whether a borrower will default on a mortgage. This is influenced by credit score, income ratio, and even the property's true market value.

Problem 3

Siloed Decision-Making

Banks historically value properties and score credit risk separately. This creates inconsistency and systemic blind spots in loan underwriting.

💡 Solution: Build an integrated ML pipeline that (1) segments neighborhoods, (2) predicts property value, and (3) classifies credit default risk — in one unified system.

M1 · System Objectives & Scope

O1

Segment neighborhoods into quality clusters using KMeans unsupervised learning on socioeconomic indicators

O2

Predict the fair market value of a property using Linear Regression on both physical and cluster features

O3

Classify applicant credit risk (default / no-default) using Random Forest classification

O4

Integrate all three models into a single sklearn Pipeline with a clean CLI user interface

O5

Simulate realistic synthetic data ($n=1000$) with controlled distributions to train and validate models

A light gray square containing the text 'M2' in a bold, white, sans-serif font.

M2

Member 2

Data Engineering & Preprocessing

Synthetic generation, distributions, cleaning, and outlier removal

M2 · Why Synthetic Data?

Privacy Safe

Real mortgage and property data contains PII. Synthetic data allows full experimentation without legal or ethical risk.

Controlled Distributions

We precisely specify statistical properties (mean, std, range) for every feature — ensuring realistic but reproducible results.

Reproducible

numpy random seed = 42 guarantees every team member gets identical data across all runs, enabling consistent model evaluation.

Scalable

n_samples = 1000 can be changed to 10,000 or 100,000 with one variable — great for testing scalability.

M2 · Feature Distributions & Statistical Design

Feature	Distribution	Rationale
sq_foot	Normal($\mu=2100$, $\sigma=450$)	Home sizes cluster around a mean; rare very large or small homes
property_age	Uniform Integer [1, 60]	Properties from newly built to 60 years old, equally likely
bedrooms	Uniform Integer [2, 6]	Most homes have 2–6 bedrooms; discrete count variable
bathrooms	Uniform Integer [1, 4]	Similar rationale to bedrooms; integer count
crime_rate	Uniform(0.1, 4.5)	Crime rate index — continuous, varies widely by area
avg_income	Normal($\mu=70000$, $\sigma=18000$)	Neighborhood income follows approx. normal distribution
school_rating	Uniform(2.0, 10.0)	School quality scores on a 2–10 scale
transit_score	Uniform(15, 85)	Transit accessibility index, continuous range
credit_score	Uniform Integer [580, 850]	580=poor, 850=perfect; subprime borrowers excluded
debt_to_income	Uniform(0.15, 0.55)	DTI ratio; >0.43 is high risk per lending standards
down_payment_pct	Uniform(0.05, 0.30)	5%–30% down; affects LTV ratio and risk

M2 · Target Variable Design & Data Cleaning

Property Price Formula (Target Variable)

$$\text{price} = (\text{sq_foot} \times 165) - (\text{property_age} \times 780) + (\text{avg_income} \times 1.4) + \varepsilon$$

where $\varepsilon \sim N(0, 12000)$ (Gaussian noise term)

$$\text{sq_foot} \times 165$$

Each additional sq ft adds \$165 to value

$$\text{property_age} \times 780$$

Each additional year of age reduces value by \$780

$$\text{avg_income} \times 1.4$$

Higher-income neighborhoods drive up prices (1.4× multiplier)

$$\varepsilon \sim N(0, 12000)$$

Stochastic noise simulates real-world valuation uncertainty

Outlier Removal: IQR Method → Remove price < (Q1 - 1.5×IQR) or price > (Q3 + 1.5×IQR)

M3

Member 3

Unsupervised Learning – KMeans

Neighborhood segmentation theory and NeighborhoodClusterer design

M3 · KMeans Clustering — Theory

Algorithm: Lloyd's KMeans

- 1 Initialize $k=3$ centroids randomly ($n_init=10$ restarts to avoid local minima)
- 2 Assign each data point to the nearest centroid (Euclidean distance)
- 3 Recompute each centroid as the mean of all points assigned to it
- 4 Repeat assignment and update until centroids converge (no change)
- 5 Select the best run (lowest inertia = sum of squared distances to centroids)

Key Concepts

Inertia:

Sum of squared distances from each point to its assigned centroid. Lower = tighter clusters.

$n_init=10$:

KMeans runs 10 times with different seeds; best result (lowest inertia) is kept.

$random_state=42$:

Ensures deterministic, reproducible centroid initialization across runs.

Euclidean Distance:

$d = \sqrt{\sum (x_i - c_i)^2}$. The distance metric used to assign points to centroids.

Convergence:

Algorithm stops when centroid positions no longer change between iterations.

M3 · NeighborhoodClusterer — Custom Transformer Design

Why inherit from BaseEstimator + TransformerMixin?

BaseEstimator → Provides `get_params()` / `set_params()` for hyperparameter management and GridSearchCV compatibility.

TransformerMixin → Provides `fit_transform()` by combining `fit()` and `transform()` — enables use inside sklearn Pipeline steps.

`fit(X, y=None)`

1. Extract demographic columns: [crime_rate, avg_income, school_rating, transit_score]
2. Fit StandardScaler on the 4 demographic features → stores μ and σ for each feature
3. Scale the features to zero mean, unit variance: $X' = (X - \mu) / \sigma$
4. Fit KMeans(`n_clusters=3`) on scaled data → learns 3 cluster centroids
5. Returns self (sklearn convention) — scaler and kmeans stored as instance attributes

`transform(X)`

1. Copy input DataFrame to avoid mutating original data
2. Extract same demographic columns from the copy
3. Apply already-fitted `scaler.transform()` — uses stored μ , σ (no re-fitting = no data leakage)
4. Predict cluster label for each row using `kmeans.predict()`
5. Add 'Neighborhood_Cluster' column; drop the 4 raw demographic columns

M3 · Why StandardScaler Before KMeans?

✗ Without Scaling

- avg_income ranges from ~34,000 – ~106,000 (scale ≈ 72,000)
- crime_rate ranges from 0.1 – 4.5 (scale ≈ 4.4)
- KMeans distance is dominated by avg_income
- School rating and crime rate have near-zero influence
- Clusters reflect income brackets only — neighborhood structure is lost

✓ With StandardScaler

- All features scaled to: mean = 0, std = 1
- crime_rate, avg_income, school_rating, transit_score treated equally
- Distance metric is fair across all dimensions
- Clusters capture multi-dimensional neighborhood quality
- Scaler stored on instance: avoids re-fitting on test data (no leakage)

Formula: $X_{\text{scaled}} = (X - \mu) / \sigma \rightarrow$ each feature has mean=0, variance=1 $\rightarrow$ KMeans distances are meaningful

M4

Member 4

Supervised Learning – Linear Regression

Regression theory, model training, and property valuation pipeline

M4 · Linear Regression — Theory

Mathematical Model

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

$\hat{y}$

Predicted property price (dependent variable)

β_0

Intercept — predicted price when all features = 0

$\beta_1 \dots \beta_n$

$x_1 \dots x_n$

Coefficients — unit change in price per unit change in feature

$x_1 \dots x_n$

$\beta_1 \dots \beta_n$

Feature values: scaled physical features + one-hot cluster IDs

Training — Ordinary Least Squares (OLS)

sklearn's LinearRegression minimizes the Residual Sum of Squares (RSS):

$$\hat{\beta} = \operatorname{argmin} \sum (y_i - \hat{y}_i)^2 = (X^T X)^{-1} X^T y \quad (\text{closed-form analytical solution})$$

M4 · Valuation Pipeline — Feature Engineering

Stage 1

NeighborhoodClusterer

unsupervised_segmentation
KMeans(k=3) on 4 demo. features
→ Neighborhood_Cluster ∈ {0,1,2}

Stage 2

ColumnTransformer

encoding_and_scaling
OneHotEncode(Cluster ID)
StandardScale(physical features)

Stage 3

LinearRegression

linear_regressor
OLS on combined features
→ predicted price \$

Features Entering LinearRegression (after ColumnTransformer):

Source	Feature(s)	Transformation	Dimensions
Cluster ID	Neighborhood_Cluster	OneHotEncoder → binary columns	3 columns (k=3)
Physical	sq_foot, property_age	StandardScaler (Z-score)	2 columns
Physical	bedrooms, bathrooms	StandardScaler (Z-score)	2 columns

A light gray square containing the text "M5" in a bold, white, sans-serif font.

M5

Member 5

Classification – Random Forest

Decision trees, ensemble learning, and credit default prediction

M5 · Decision Trees — Building Blocks

Decision Node

A feature split: e.g., 'Is credit_score < 680?' Each node divides data into two branches based on a threshold.

Leaf Node

Terminal node containing the majority class (default=1 or no-default=0) for all samples reaching that node.

Gini Impurity

$Gini = 1 - \sum p_i^2$ → Measures how 'mixed' a node is. 0 = perfectly pure (one class only). The tree splits to minimize Gini.

Information Gain

$IG = Gini(parent) - [\text{weighted avg } Gini(children)]$. Tree picks the split that maximizes information gain.

Max Depth = 6

Limits tree depth to 6 levels. Prevents overfitting by stopping the tree from memorizing training noise.

Pruning Effect

max_depth=6 acts as pre-pruning. Deeper trees overfit; shallower trees generalize better to unseen data.

M5 · Random Forest — Ensemble Theory

Ensemble Principle: Train 100 independent decision trees; combine by majority vote (classification) or average (regression).

Bagging (Bootstrap Aggregation)

Each tree is trained on a random bootstrap sample (with replacement) of the training data. ~63% unique samples per tree. Reduces variance.

Majority Vote

Final prediction = class voted by most trees. For probabilities, `predict_proba()` returns the fraction of trees voting for each class.

Bias-Variance Tradeoff

Individual trees = low bias, high variance. Averaging 100 trees = low bias, dramatically lower variance. Best of both worlds.

Feature Randomness

At each split, only a random subset of $\sqrt{p}$ features are considered. This decorrelates trees — no single feature dominates every tree.

Why 100 Trees?

`n_estimators=100` provides stable estimates. More trees → lower variance but slower. 100 is standard for most tabular datasets.

Out-of-Bag Error

Since ~37% of samples aren't used per tree, they serve as built-in validation. sklearn's OOB score can estimate generalization error.

M5 · Credit Default Risk Model — Design

Input Features (3 variables):

credit_score

Range: 580–850

Primary indicator of borrower reliability and payment history

down_payment_pct

Range: 0.05–0.30

Higher down payment = lower LTV = lower default risk

predicted_house_price

Range: Dynamic

From valuation pipeline — links property value to affordability

Synthetic Label Formula

```
risk = (DTI × 7.5)  
      - (credit_score / 155)  
      - (down_pmt × 3.5)
```

```
p_default = sigmoid(risk)  
default = p > Uniform(0,1)
```

Decision Threshold = 0.25

```
predict_proba()[ :, 1] ≥ 0.25  
→ DENIED   |   < 0.25 → APPROVED
```

Classification Pipeline: StandardScaler → RandomForestClassifier(n_estimators=100, max_depth=6)
StandardScaler normalizes features before RF — improves convergence even for tree-based methods.

M6

Member 6

System Integration & Pipeline

How all components connect into one deployable inference system

M6 · sklearn Pipeline — Theory & Benefits

sklearn Pipeline chains transformers and an estimator. Calling `pipeline.fit(X, y)` automatically calls `fit_transform()` on each transformer, then `fit()` on the final estimator. `pipeline.predict(X)` chains `transform()` → `predict()`.

No Data Leakage

Transformers are fit on training data only; test data is transformed using stored parameters. Pipeline enforces this automatically.

Single Predict Call

One call to `pipeline.predict(X_new)` handles all preprocessing and inference for unseen data — no manual preprocessing needed.

GridSearchCV Ready

The entire pipeline can be passed to GridSearchCV. Hyperparameters referenced as 'step__param' (e.g., 'linear_regressor__fit_intercept').

Reproducibility

All fitted parameters (scaler μ/σ , KMeans centroids, regression weights) are stored in one pipeline object — serializable with joblib.

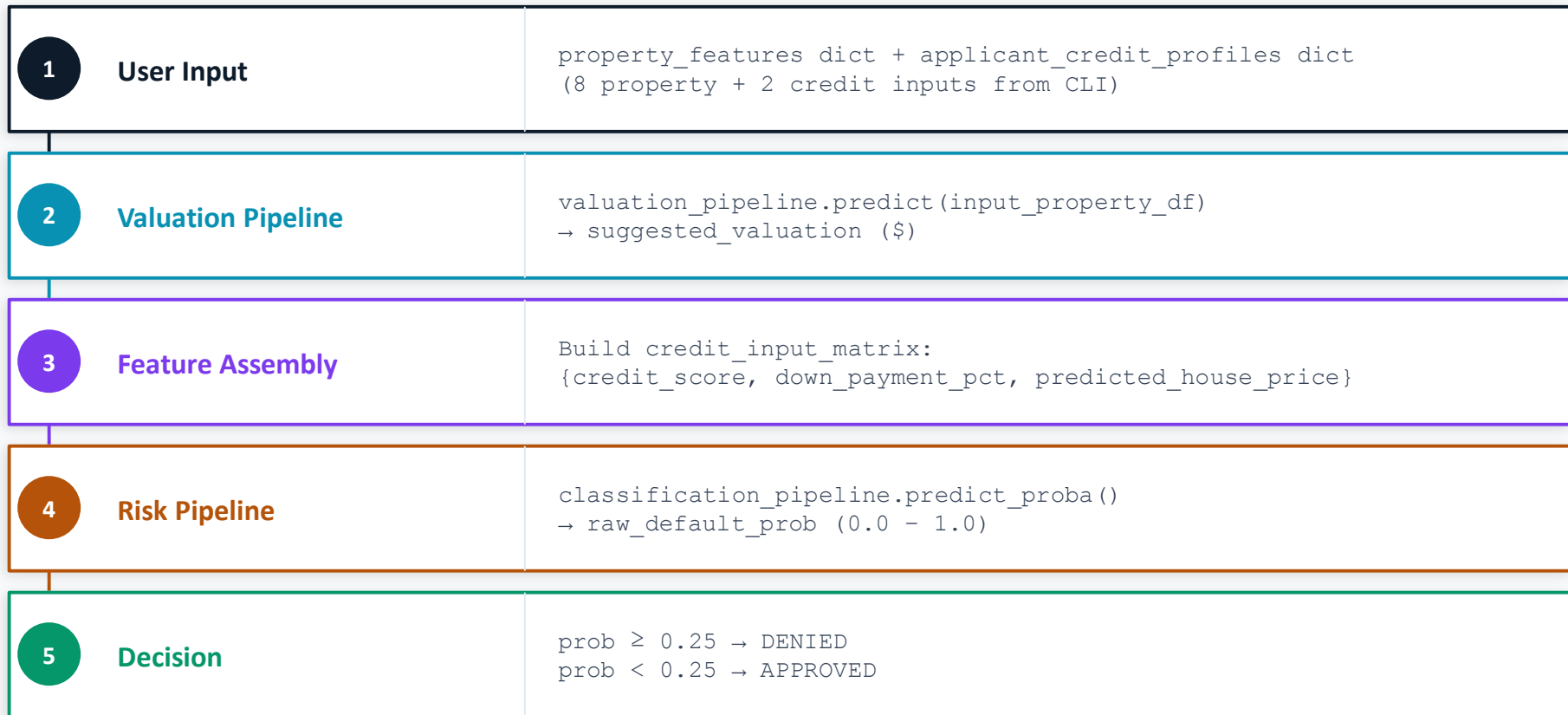
Error Prevention

Without Pipeline, a developer might accidentally call `scaler.fit_transform()` on test data. Pipeline makes this mistake structurally impossible.

Deployment Ready

The fitted pipeline object is the entire model artifact. Deploy it to production, and it handles raw input → prediction in one step.

M6 · End-to-End Prediction Flow



The logo consists of a light gray square containing the text 'M7' in a bold, white, sans-serif font.

M7

Member 7

Results, Evaluation & Conclusions

Model performance, theoretical validation, and key learnings

M7 · Model Evaluation Metrics — Theory

Regression Metrics (Valuation Model)

MAE

$$(1/n) \sum |y_i - \hat{y}_i|$$

Average absolute error in \$ — easy to interpret, robust to outliers

RMSE

$$\sqrt{[(1/n) \sum (y_i - \hat{y}_i)^2]}$$

Penalizes large errors more — good for catching extreme mispricings

R² Score

$$1 - SS_{\text{res}}/SS_{\text{tot}}$$

Fraction of variance explained. R²=1 perfect, R²=0 = naive mean model

Classification Metrics (Credit Risk Model)

Accuracy

$$(TP+TN) / (TP+TN+FP+FN)$$

Overall correct predictions — misleading for imbalanced classes

Precision

$$TP / (TP + FP)$$

Of predicted defaults, how many actually defaulted?

Recall

$$TP / (TP + FN)$$

Of actual defaults, how many did we catch? (critical for lenders)

AUC-ROC

Area under ROC curve

Discrimination ability across all thresholds — 0.5=random, 1.0=perfect

M7 • Key Learnings & Theoretical Contributions

UNSUPERVISED ML

KMeans as Feature Engineering

Using clustering as a preprocessing step is a powerful technique. Neighborhood cluster IDs serve as latent quality features that improve regression accuracy more than the raw demographic values would.

FEATURE ENGINEERING

Bridging Two Pipelines

The predicted_house_price from the regression pipeline becomes an input to the classification pipeline. This cross-pipeline feature linkage is a real-world pattern in production ML systems.

SKLEARN DESIGN

Custom Transformers Enable Modularity

Implementing BaseEstimator + TransformerMixin allows our clustering step to be swapped, tuned, or replaced without changing any other code. This is the open/closed principle applied to ML.

RISK MANAGEMENT

Conservative Thresholding

Setting the default threshold at 0.25 (rather than standard 0.50) reflects asymmetric costs: a missed default is far more costly to a lender than a rejected good applicant.

Future Improvements & Discussion

Real Data Integration

Replace synthetic data with actual MLS property listings and HMDA mortgage data for production accuracy.

Hyperparameter Tuning

Use GridSearchCV or RandomizedSearchCV on pipeline params: `n_clusters`, `n_estimators`, `max_depth`, `threshold`.

Advanced Models

Replace LinearRegression with GradientBoostingRegressor or XGBoost for non-linear valuation. Try LightGBM for credit risk.

Model Explainability

Add SHAP values to explain each prediction — critical for regulatory compliance in lending (ECOA, FCRA).

Cross-Validation

Implement k-Fold CV (currently absent) to get unbiased error estimates and confidence intervals on all metrics.

Production Deployment

Serialize pipeline with joblib, expose via FastAPI endpoint, containerize with Docker for real-time underwriting.